

Registro de Evaluación

Nombre(s): Jair Joonpool

Apellidos: Olvea Aliaga

Código: 22200169

Fecha: 20/06/2026

1 Examen de Desarrollo de Sistemas Móviles - Nivel Básico - Intermedio

Información General

Puntaje Total: 20 puntos

Duración: 2 horas + 10 min para entrega

Modalidad: Preguntas Abiertas y Desarrollo de Código.

Instrucciones: Resuelva el examen de forma individual y a mano. Al finalizar, digitalice todas las páginas en un único archivo PDF denominado prueba1.pdf y súbalo al repositorio: github.com/illarek-lab/mobile_pruebas

Dentro del repositorio cree un branch C<codigo_estudiante> con un directorio con el formato: C<codigo_estudiante> y coloque en su interior el archivo: prueba1.pdf y realice su PR. Ejemplo: C20251234/prueba1.pdf

2 1. Análisis de Arquitectura y Concurrencia (8 puntos)

Valor: 2 pts c/u

1. El Patrón Repositorio como SSOT

Explique por qué el Repositorio se considera la "Única Fuente de Verdad" (SSOT) y una "fachada" en la arquitectura. Detalle cómo coordina los datos entre la base de datos local (Room) y la API remota (Retrofit) para que la UI no sepa de dónde vienen los datos.

El Patrón Repository se considera la Single Source of Truth porque centraliza toda la gestión de datos de la aplicación en un único punto de acceso. Dentro de la arquitectura MVVM utilizada en el curso, la capa de UI nunca accede directamente a Room ni a Retrofit, sino que interactúa únicamente con el ViewModel, el cual delega la obtención de datos al Repository.

Además, actúa como una fachada porque abstrae la complejidad de las diferentes fuentes de datos y expone una interfaz uniforme para el resto de la aplicación. Su funcionamiento es:

- Room presenta la fuente de datos local
- Retrofit representa la fuente de datos remota
- El Repository coordina ambas fuentes
- Cuando los datos remotos son obtenidos mediante Retrofit, estos son persistidos en Room
- Room emite automáticamente los cambios mediante Flow <List<T>>, actualizando la interfaz de forma reactiva.

Registro de Evaluación

Nombre(s): Jair Joopool

Apellidos: Olvea Alraga

Código: 22200169

Fecha: 20/06/2026

1 Examen de Desarrollo de Sistemas Móviles - Nivel Básico - Intermedio

Información General

Puntaje Total: 20 puntos

Duración: 2 horas + 10 min para entrega

Modalidad: Preguntas Abiertas y Desarrollo de Código.

Instrucciones: Resuelva el examen de forma individual y a mano. Al finalizar, digitalice todas las páginas en un único archivo PDF denominado prueba1.pdf y súbalo al repositorio: github.com/illarek-lab/mobile_pruebas

Dentro del repositorio cree un branch C<codigo_estudiante> con un directorio con el formato: C<codigo_estudiante> y coloque en su interior el archivo: prueba1.pdf y realice su PR. Ejemplo: C20251234/prueba1.pdf

2 1. Análisis de Arquitectura y Concurrencia (8 puntos)

Valor: 2 pts c/u

1. El Patrón Repositorio como SSOT

Explique por qué el Repositorio se considera la 'Única Fuente de Verdad' (SSOT) y una 'fachada' en la arquitectura. Detalle cómo coordina los datos entre la base de datos local (Room) y la API remota (Retrofit) para que la UI no sepa de dónde vienen los datos.

El patrón Repository se considera la Single Source of Truth porque centraliza toda la gestión de datos de la aplicación en un único punto de acceso. Dentro de la arquitectura MVVM utilizada en el curso, la capa de UI nunca accede directamente a Room ni a Retrofit, sino que interactúa únicamente con el ViewModel, el cual delega la obtención de datos al Repository.

Además, actúa como una fachada porque abstrae la complejidad de las diferentes fuentes de datos y expone una interfaz uniforme para el resto de la aplicación.

Su funcionamiento es:

- Room presenta la fuente de datos local
- Retrofit representa la fuente de datos remota
- El Repository coordina ambas fuentes
- Cuando los datos remotos son obtenidos mediante Retrofit, estos son persistidos en Room
- Room emite automáticamente los cambios mediante Flow<List<T>>, actualizando la interfaz de forma reactiva.

2. Corrutinas y Structured Concurrency

Defina qué es el `viewModelScope` y por qué es fundamental para la gestión de memoria. Explique qué sucede con una petición de red iniciada en este scope si el usuario decide cerrar la pantalla antes de recibir la respuesta.

`viewModelScope` es un `CoroutineScope` asociado al ciclo de vida del `ViewModel`. Su finalidad es ejecutar tareas asíncronas respetando el principio de `Structured Concurrency`, permitiendo que todas las corrutinas hijas dependan de un scope padre. Su importancia es:

- ~~Evitar~~ - Evitar bloqueos del Main Thread
- Gestionar automáticamente la cancelación de tareas
- Prevenir Memory Leaks
- Liberar recursos cuando el `ViewModel` deja de existir.

Si una petición de red realizada mediante `Retrofit` se ejecuta dentro de `viewModelScope` y el usuario abandona la pantalla antes de recibir la respuesta:

1. El `viewModel` es destruido
2. Se ejecuta el proceso de limpieza del ciclo de vida.
3. Las corrutinas asociadas son canceladas automáticamente.
4. La petición deja de consumir recursos.

3. Flujos Reactivos (Cold vs. Hot)

Diferencie técnicamente un `Flow` (frío) de un `StateFlow` (caliente). ¿Por qué para el manejo de estados de la interfaz de usuario se prefiere `StateFlow` sobre un `Flow` convencional?

`Flow` (Cold Stream)
↳ es un flujo frío

- No produce valores hasta que existe un colector
- Cada observador genera una nueva ejecución.
- No conserva el último valor emitido.

`StateFlow` (Hot Stream)
↳ es un flujo caliente

- Permanece activo incluso sin observadores
- Mantiene un estado actual
- Reemite automáticamente el último valor a nuevos observadores

Para la gestión de estados se utiliza `StateFlow` porque `Compose` necesita reaccionar a cambios de estado de forma continua, cuando:

1. `StateFlow` cambia
2. `Compose` detecta el cambio
3. Se produce la recomposición automática de la interfaz.

Por ello `StateFlow` es la solución recomendada para representar estados como

4. Inyección de Dependencias (DI) Manual

`Loading`, `Success` y `Error`.

Describe el mecanismo de "DI Manual" utilizado en los laboratorios mediante la clase `Application`. Explique para qué sirve la palabra reservada `by lazy` en la inicialización de los objetos de infraestructura (como la BD o el Repositorio).

En los laboratorios se emplea `Dependency Injection Manual` utilizando una clase que hereda de `Application`.

Esta clase se comporta como un contenedor global de dependencias y crea instancias únicas de:

- Room Database - DAO - Repository - etc.

Las `Activities` o `ViewModels` obtienen dichas dependencias mediante:
`application as My Application`

Esto garantiza que todos los componentes compartan la misma instancia del repositorio.

La palabra reservada `by lazy` implementa el patrón de inicialización diferida

Beneficios:

- El objeto se crea únicamente cuando es utilizado
- Reduce el consumo de memoria
- Evita inicializaciones innecesarias
- Garantiza una única instancia compartida

3 II. Sección de Completar (4 puntos)

Instrucción: Escriba el término técnico exacto que completa la oración basándose en el texto de las sesiones.

1. El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow <T>
2. El operador Dispatchers.Default permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos.
3. Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo location
4. La función application de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".
5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama suspend Coroutine
6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml
7. En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL (Hardware Abstraction Layer)
8. El operador de Flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es combine

4 III. Desarrollo de Código y Lógica Personalizada (5 puntos)

Instrucción: Escriba el código en Kotlin siguiendo las convenciones de PascalCase para clases y camelCase para variables.

1. (2 pts) Repositorio y DI

Escriba el código de una clase UsuarioRepository que reciba por constructor un DAO. Luego, muestre cómo se instanciaría este repositorio dentro de una clase que extienda de Application usando el patrón by lazy.

```
class UsuarioRepository {  
    private val usuarioDao: UsuarioDao  
} {  
    fun obtenerUsuarios() = usuarioDao.obtenerUsuarios()  
}
```

```
class MyApplication : Application() {  
    private val database by lazy {  
        AppDatabase.getDatabase(this)  
    }  
    val usuarioRepository by lazy {  
        UsuarioRepository(database.usuarioDao())  
    }  
}
```


2. (3 pts) Lógica Dinámica de Identidad

- a) Cree una data class Usuario con los atributos: nombre (String), apellidos (String) y edad (Int?).
- b) Cree un objeto con sus datos reales.
- c) Escriba una lógica que calcule el "Puntaje de Identidad": la suma del número de letras de su nombre + el número de letras de sus apellidos.
- d) Escriba el código para imprimir el mensaje:

Usuario: [Nombre Completo], tu puntaje de identidad es: [Resultado]

""

- e) Use el operador Elvis (?) para asegurar que el puntaje sea 0 si los datos fuesen nulos. ""

```
① data class Usuario(  
    val nombre: String,  
    val apellidos: String,  
    val edad: Int?  
)
```

```
② val usuario = Usuario(  
    nombre = "Jair",  
    apellidos = "Olvea Alraga",  
    edad = 22  
)
```

③ ④ y ⑤

```
val letrasNombre =  
    usuario.nombre?.length ?: 0
```

```
val letrasApellidos =  
    usuario.apellidos  
        ?.replace(" ", "")  
        ?.length ?: 0
```

```
val puntajeIdentidad = letrasNombre + letrasApellidos
```

```
println("usuario: ${usuario.nombre} ${usuario.apellidos}, " +  
    "tu puntaje de identidad es: $puntajeIdentidad")
```

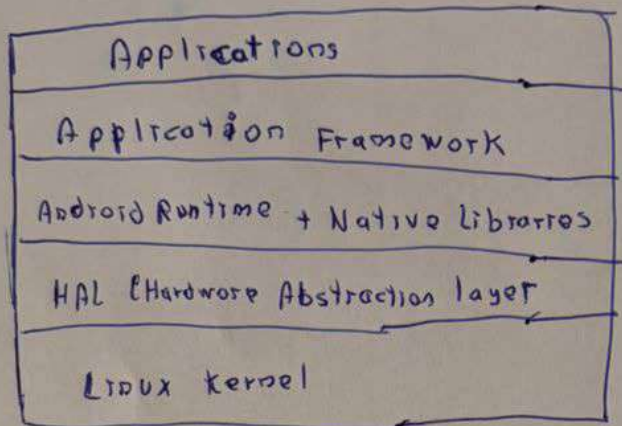
```
)
```

5 IV. Arquitectura del AOSP Completa (3 puntos)

Instrucción: La comprensión de la pila de software es vital para el desarrollo nativo.

1. Gráfico (1.5 pts)

Dibuje el diagrama de las 5 capas de la arquitectura del Android Open Source Project (AOSP) en el orden jerárquico correcto.



2. Identificación Total (1.5 pts)

Nombre y describa brevemente la función de cada una de las 5 capas que graficó, explicando específicamente qué componentes residen en la capa de Android Runtime (ART).

a. Applications

Contiene las aplicaciones instaladas por el usuario y las aplicaciones del sistema. Estas aplicaciones consumen las APIs públicas proporcionadas por Android.

b. Application Framework

Proporciona los servicios de alto nivel utilizados por las aplicaciones Android. Como:

- Activity Manager
- Package Manager
- Notification Manager
- etc.

c. Android Runtime

Es el entorno de ejecución encargado de transformar el bytecode de la aplicación en instrucciones ejecutables para el procesador del dispositivo. Dentro de esta capa reside:

- Android Runtime
- Core Java Libraries
- Garbage Collector
- Gestión de memoria
- Ejecución de bytecode DEX

d. HAL

Funciona como un puente estándar entre Android y los controladores de hardware. Permite que componentes como GPS, cámara, sensores, etc. que puedan utilizarse sin depender de la implementación específica del fabricante.

e. Linux kernel

Es la capa más baja del sistema operativo Android.

Funciones:

- Gestión de memoria
- Gestión de procesos
- seguridad
- Drivers de hardware
- Comunicación con dispositivos físicos.

Nota para el Estudiante

Se evaluará la precisión terminológica (por ejemplo, no confundir Thread con Coroutine) y el respeto a la jerarquía de capas de la arquitectura Android moderna.